DIGITALNI VLSI SISTEMI 2025/2026 – PROJEKTNI ZADATAK

UBRZANJE FILTRIRANJA SLIKE

1. Opis sistema

Potrebno je realizovati sistem kojim se implementira linearno, prostorno invarijantno, filtriranje slike korišćenjem proizvoljne prostorne maske maksimalne veličine 9x9.

Sistem se sastoji iz procesorskog dela (Zynq7), DMA kontrolera, AXI Timer komponente i komponente akceleratora filtriranja slike (**acc_image_filter**).

Ulazna slika (**img**) je zadata u **uint8** formatu i nalazi se u binarnom fajlu na **host** računaru. Slika se može preneti u DDR memoriju na razvojnom sistemu korišćenjem **xdb** komande **mwr**. Nakon toga potrebno je filtrirati zadatu sliku korišćenjem prostornog filtra zadatih dimenzija i koeficijenata filtriranja. Detalji algoritma su navedeni u narednim odeljcima.

Računanje se najpre obavlja softverski korišćenjem funkcije **ImageFilterSW** implementirane na procesorskom delu sistema. Ulazni argumenti funkcije su pokazivač na ulaznu sliku **DataIn**, pokazivač na izlazni bafer **DataOut** kao i parametri procesiranja koji se sadrže u strukturi **ProcessingParams**. Ova struktura ima polja koja definišu širinu i visinu ulazne slike (**ImgW**, **ImgH**), radijus filtra (**FilterRadius**) prostorna maska je dimenzija 2·FilterRadius+1, niz vrednosti koeficijenata (**FilterCoeffs**), faktor skaliranja koeficijenata (**FilterCoeffsScale**), način tretiranja ivičnih elemenata (**Border**) koji ima efekat jedino ukoliko se implementira bonus deo projekta.

Procesiranje se može ubrzati korišćenjem komponente hardverskog akceleratora. Potrebno je realizovati funkciju **ImageFIlterHW** kojom se omogućava korišćenje hardverskog akceleratora **acc_image_filter**. Komponentu akceleratora je potrebno realizovati u okviru projekta prema specifikacijama koje su date u narednim odeljcima. Argumenti funkcije su identični kao i argumenti funkcije kojom se realizuje softversko procesiranje. U okviru funkcije je potrebno konfigurisati komponentu akceleratora kao i komponentu DMA kontrolera. Komponentu DMA kontrolera konfigurisati tako da se elementi ulazne slike šalju preko AXI Stream interfejsa u modul akceleratora i da se izlazni podaci iz akceleratora prihvataju preko AXI Stream interfejsa i upisuju u memoriju izlaznog bafera.

Korišćenjem komponente AXI Timer-a odrediti vreme izvršavanja funkcija **ImageFilterSW** i **ImageFilterHW** u mikrosekundama. Uporediti brzine procesiranja i odrediti faktor ubrzanja koje se postiže korišćenjem modula hardverskog akceleratora.

Implementirati funkciju **CheckData** kojom se verifikuje poklapanje rezultata softverskog i hardverskog procesiranja.

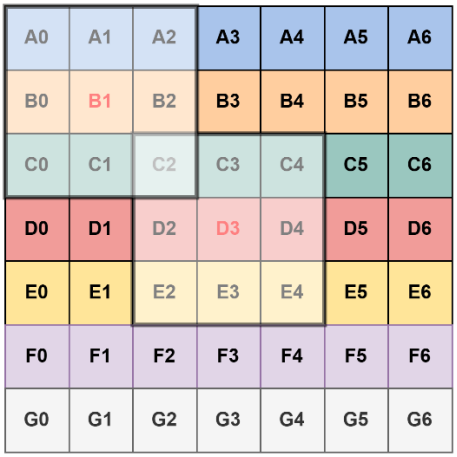
Rezultat procesiranja snimiti u odgovarajući fajl na host računaru, korišćenjem **mrd** komande, i prikazati ga korišćenjem datih **Python** skripiti. Uporediti rezultate softverskog i hardverskog procesiranja sa rezultatim procesiranja u **Python**-u i verifikovati ispravnost primenjenog pristupa. Korišćenjem date **Python** skripte se mogu proizvoljne ulazne slike učitati, skalirati i

upisati u binarni fajl kako bi se odatle prenele u memoriju i procesirale korišćenjem kreiranog akceleratora. Prilkom testiranja vremena procesiranja koristiti sliku **lena_512.bin** i **lena_128.bin**.

Omogućiti da se, bez ponovnog pokretanja programa, nakon završetka procesiranja trenutne slike pređe na procesiranje naredne slike sa novim parametrima procesiranja. Demonstrirati funkcionalnost sistema testiranjem različitih parametara procesiranja kao i različitih ulaznih slika.

2. Opis algoritma

Elementi izlazne slike predstavljaju težinsku sumu kvadratnog lokalnog susedstva zadate veličine oko piksela koji se trenutno procesira. Na slici su prikazana lokalna susedstva koja se koriste prilikom filtriranja piksela B1 i piksela D3 u slučaju filtra radijusa 1.



Slika 2.1. Lokalna susedstva za izračunavanje izlazne vrednosti prilikom filtriranja piksela B1 i piksela D3.

Koeficijenti filtra predstavljaju niz od $\mathbf{(2 \cdot FilterRadius + 1)^2}$ brojeva, tj definisan je po jedan težinski faktor za svaki piksel iz lokalnog susedstva koje se posmatra pri filtriranju. Izlazna vrednost predstavlja težinsku sumu svih elemenata iz loklanog susedstva trenutno posmatranog piksela. U slučaju filtra radijusa 1 potrebno je specificirati niz **W** od 9 težinskih koeficijenata. Izlazne vrednosti $B1_{out}$ i $D3_{out}$ koje predstavljaju rezultat filtriranja piksela B1 i D3 ulazne slike se određuju na sledeći način:

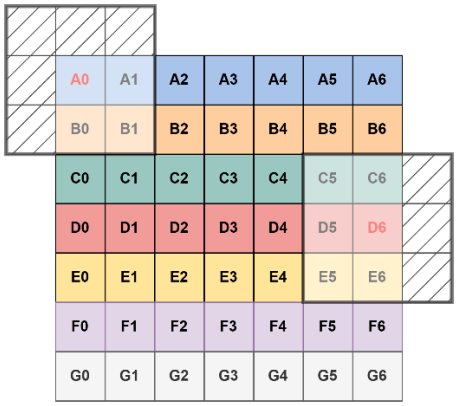
$$B1_{out} = W_1 \cdot A0 + W_2 \cdot A1 + W_3 \cdot A2 + W_4 \cdot B0 + W_5 \cdot B1 + W_6 \cdot B2 + W_7 \cdot C0 + W_8 \cdot C1 + W_9 \cdot C2$$

$$D3_{out} = W_1 \cdot C2 + W_2 \cdot C3 + W_3 \cdot C4 + W_4 \cdot D2 + W_5 \cdot D3 + W_6 \cdot D4 + W_7 \cdot E2 + W_8 \cdot E3 + W_9 \cdot E4$$

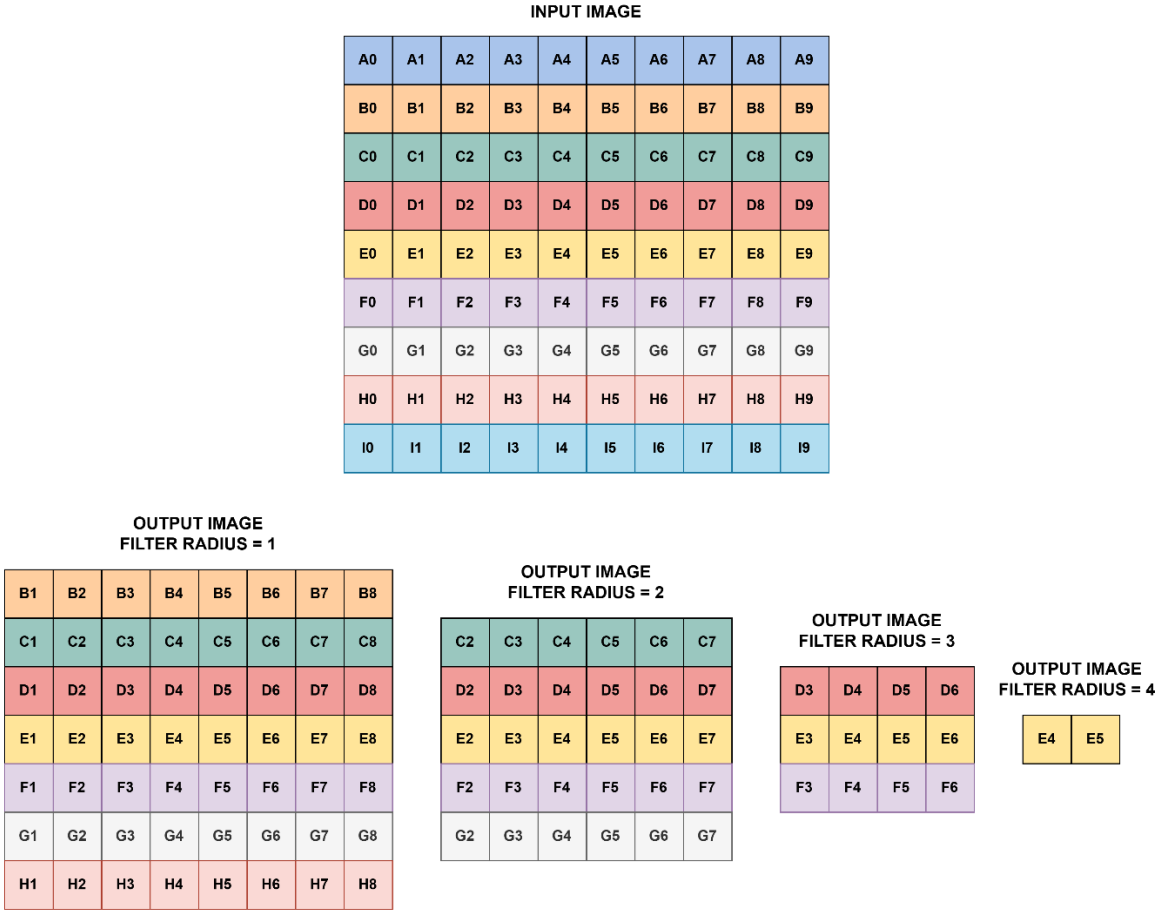
3. Ivični uslovi

Obratiti pažnju da nemaju svi pikseli potpuno definisano lokalno susedstvo kao što je prikazano na slici 3.1. Zapravo svi pikseli čije je odstojanje od okvira slike manje od radijusa filtra nemaju potpuno definisano lokalno susedstvo. U primeru sa slike 2.1. za filtar radijusa 1

to su redovi A i G kao i pikseli sa indeksima 0 i 6. Ukoliko se radijus poveća na 2 redovi A, B, F i G kao i pikseli sa indeksima 0, 1, 5 i 6 nemaju potpuno definisano lokalno susedstvo. Jedan pristup rešavanju ovog problema je da se za ove piksele ne računaju izlazne vrednosti. **U tom slučaju je izlazna slika manjih dimenzija od ulazne slike za 2·FilterRadius po obe dimenzije, kao što je prikazano na slici 3.2.** U okviru ovog projekta je potrebno realizovati ovaj pristup.

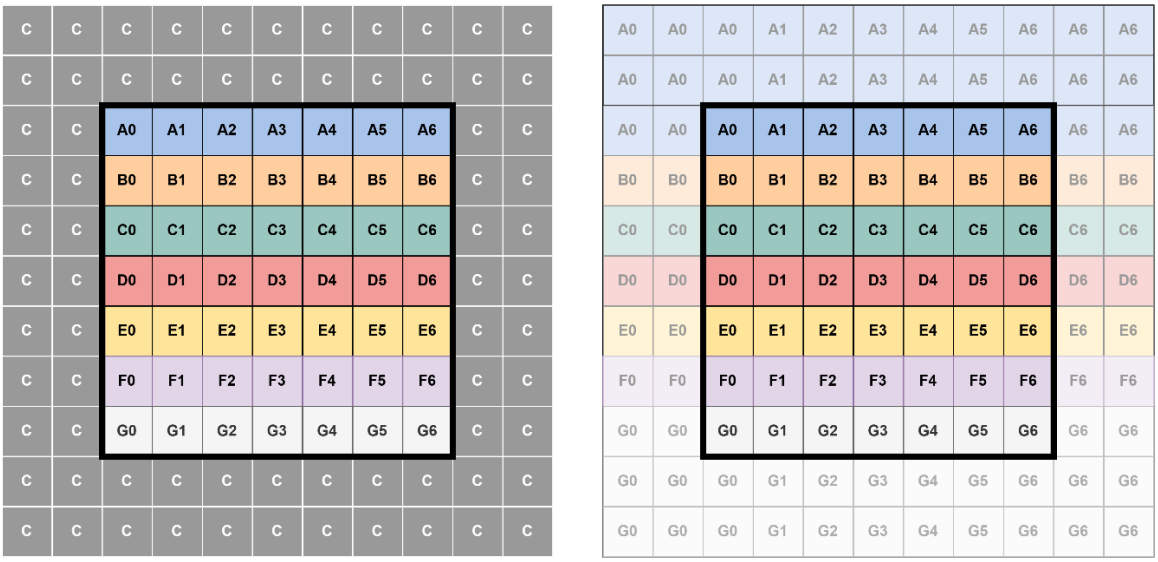


Slika 3.1. Ukoliko se piksel nalazi na rastojanju manjem od radijusa filtra od ivica slike, nema potpuno definisano lokalno susedstvo



Slika 3.2. Smanjivanje izlazne slike usled nepotpunog lokalnog susedstva za različite vrednosti radijusa filtra

[BONUS 1 → +5 poena] Problem sa prethodnim pristupom je što se prilikom filtriranja dimenzije izlazne slike smanjuju u odnosu na ulaznu, kao što je prikazano na slici 3.2. Ukoliko se slika filtrira više puta to može dovesti do značajnog smanjenja dimenzija. Kako bi se obezbedile iste dimenzije izlazne i ulazne slike potrebno je definisati neke vrednosti za elemente lokalnog susedstva koji ispadaju van okvira slike. Na slici 3.3. su prikazana dva pristupa rešenju ovog problema. Jedan način, prikazan na slici 3.3. a) je da se za sve elemente lokalnog susedstva koji se nalaze van okvira slike podrazumeva neka konstanta vrednost iz opsega vrednosti piskela, odnosno 0 do 255 u ovom slučaju. Najčešće se uzima vrednost 0 ali bilo koja vrednost iz opsega vrednosti piksela dolazi u obzir. Drugi način, prikazan na slici 3.3. b) podrazumeva da se za vrednosti lokalnog susedstva koje se nalaze van okvira slike uzima vrednost najbližeg piksela unutar okvira slike. U okviru slike 3.3. su prikazana proširenja nepohodna za filtar radijusa 2, odnosno dimenzija 5x5.



a) proširenje konstantom vrednošću

b) proširenje ponavljanjem graničnih piksela

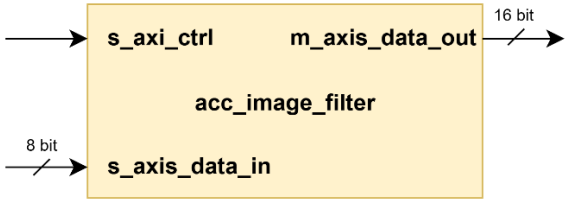
Slika 3.3. Pristupi definisanju elemenata lokalnog susedstva koji se nalaze van okvira ulazne slike

Za bonus poene potrebno je omogućiti da izlazna slika ima iste dimenzije kao i ulazna pri čemu je potrebno implementirati oba načina proširenja. Način proširenja se definiše parametrom **Border**. **[BONUS 1]**

4. Arhitektura modula akceleratora

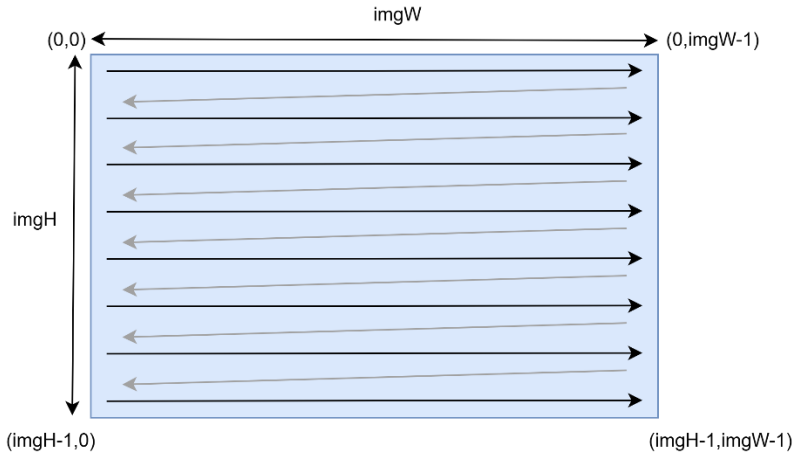
Interfejs modula akceleratora filtriranja slike je prikazan na slici 4.1. Ulazni pikseli se upisuju u akcelerator preko AXI4-Stream interfejsa **s_axis_data_in** kod koga je linija podataka (tdata) široka 8 bita, pošto prihvata piksele u **uint8** formatu. Rezultati se nakon obrade šalju preko izlaznog AXI4-Stream interfejsa **m_axis_data_out**. Širina podataka izlaznog interfejsa je 16 bita pošto izlazni podaci mogu biti ili 8-bitni neoznačeni brojevi u **uint8** formatu ili 16 bitni označeni brojevi kod kojih se 9 bita koristi za ceo deo a 7 bita za razlomljeni deo. Ukolko je izlaz 16-bitni onda se u jednoj transakciji šalje jedan podatak dok se u slučaju 8-bitnog izlaza u jednoj transakciji mogu poslati dva izlazna podatka. Konfiguracija akceleratora se nalazi u

registrima koji su definisani u odeljku 5. Ovim registrima se pristupa preko AXI4-Lite interfejsa **s_axi_ctrl**.



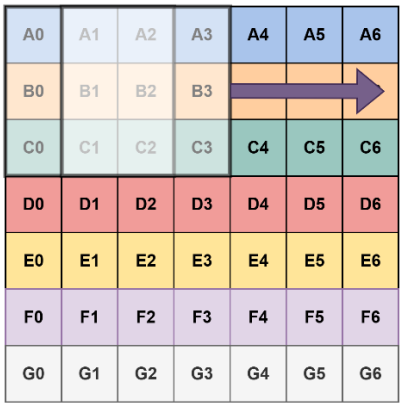
Slika 4.1. Interfejs modula akceleratora

Proces filtriranja se obavlja računanjem težinske sume piksela koji se nalaze u lokalnom susedstvu procesiranog piksela na način koji je prikazan na slici 2.1 i ilustrovan izrazima na kraju odeljka 2. Procesiranje se obavlja s leva na desno, odozgo na dole i prikazano je na slici 4.2. Odnosno ako se pogleda slika 2.1. u akcelerator najpre stižu pikseli reda A redom od 0 do kraja reda, zatim pikseli reda B i tako dalje.



Slika 4.2. Redosled procesiranja

Prilikom procesiranja piksela jednog reda, prozor koji predstavlja lokalno susedstvo se pomera s leva na desno duž tog reda kao na slici 4.3.



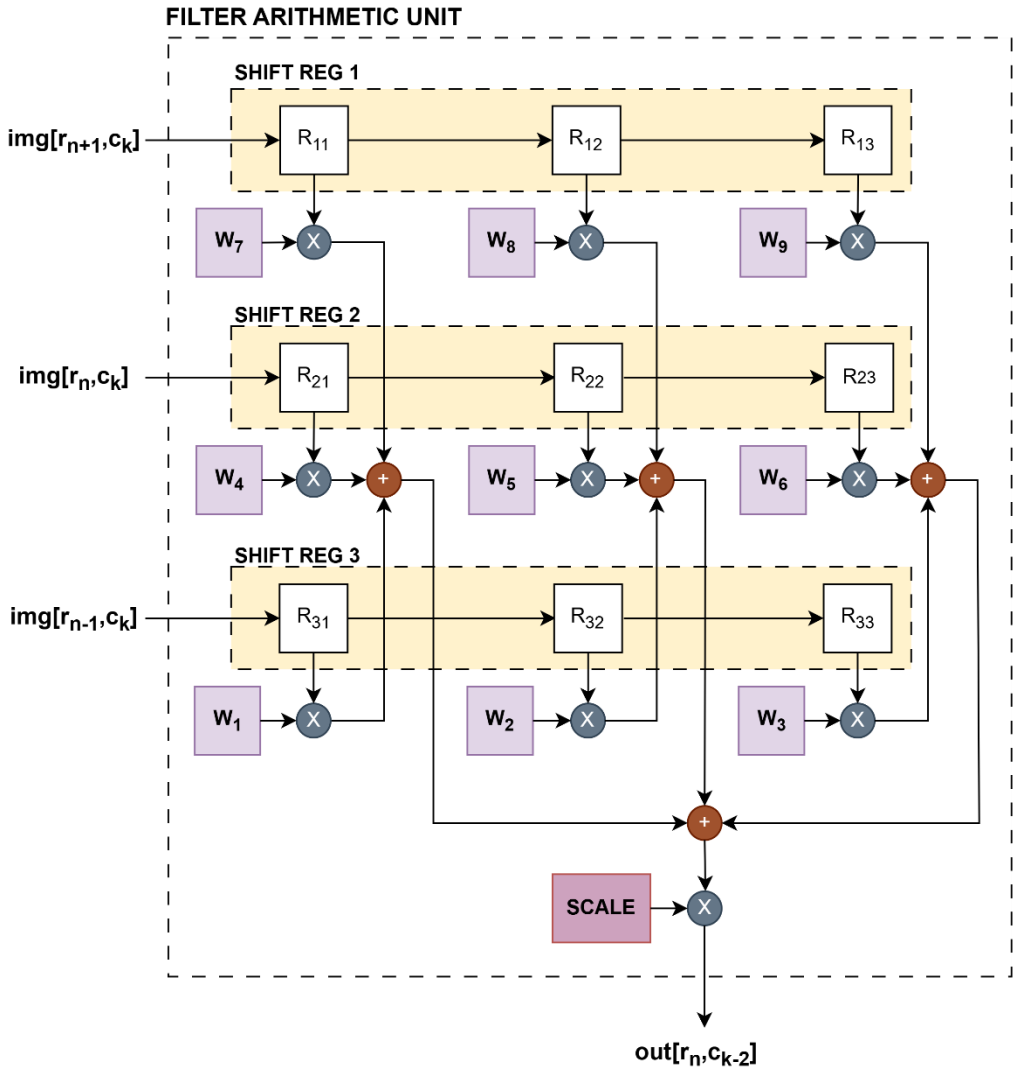
Slika 4.3. Pomeranje lokalnog susedstva

Izrazi za određivanje izlaza koji odgovaraju pikselima B1 i B2 su:

$$B1_{out} = W_1 \cdot A0 + W_2 \cdot A1 + W_3 \cdot A2 + W_4 \cdot B0 + W_5 \cdot B1 + W_6 \cdot B2 + W_7 \cdot C0 + W_8 \cdot C1 + W_9 \cdot C2$$

$$B2_{out} = W_1 \cdot A1 + W_2 \cdot A2 + W_3 \cdot A3 + W_4 \cdot B1 + W_5 \cdot B2 + W_6 \cdot B3 + W_7 \cdot C1 + W_8 \cdot C2 + W_9 \cdot C3$$

Iz ovih izraza se može zaključiti da je centralnu aritmetičku jedinicu akceleratora potrebno organizovati u vidu K pomeračkih registara, gde K predstavlja maksimalnu visinu lokalnog susedstva dok širine pojedinačnih registara predstavljaju maksimalnu širinu lokalnog susedstva koje se procesira. Primer aritmetičke jedinice akceleratora za lokalno susedstvo dimenzija 3x3 je prikazan na slici 4.4.

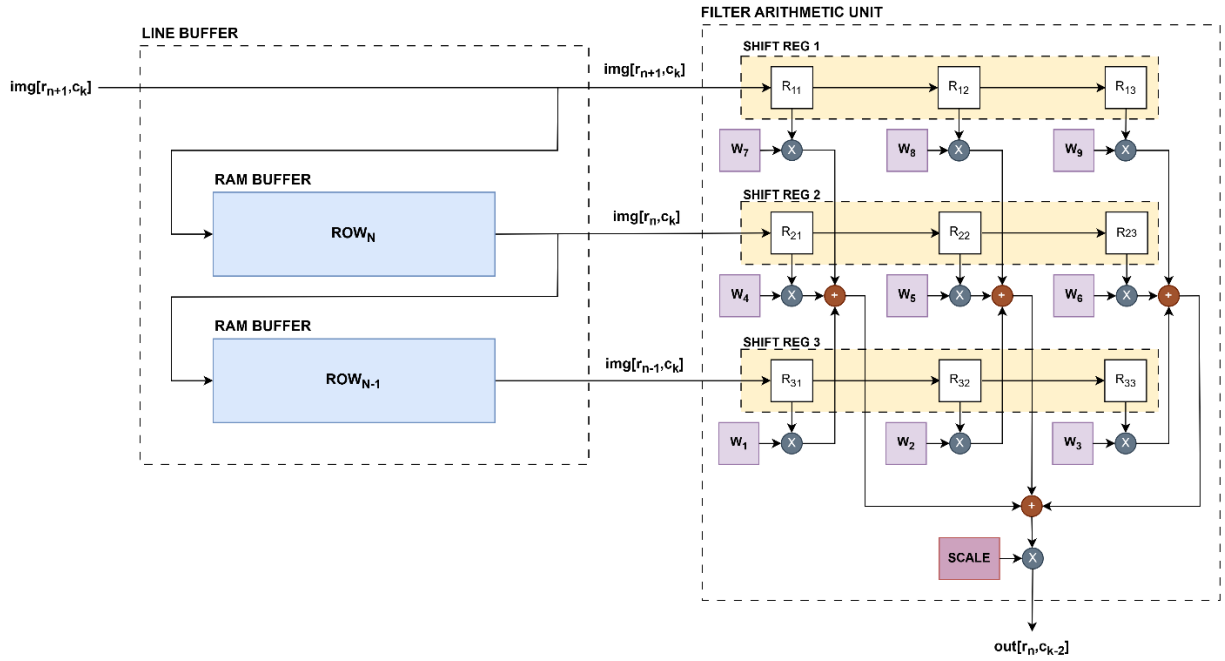


Slika 4.4. Aritmetička jedinica akceleratora za računanje izlaznog piksela na osnovu lokalnog susedstva dimenzija 3x3

Za određivanje rezultata filtriranja odgovarajućeg piksela potrebno je da svi elementi lokalnog susedstva budu prisutni u akceleratoru. Ovo podrazumeva **FilterRadius** redova slike koji su već ranije stigli, ali i **FilterRadius** redova slike koji tek treba da budu upisani u akcelerator. Dakle, broj redova slike koje je portrebno čuvati zavisi od veličine filtra i iznosi 2·FilterRadius.

Kako ova količina podataka može biti dosta velika, korišćenje registara za ovu namenu ne dolazi u obzir. Dakle, ove piksele je potrebno na neki način smestiti u BRAM blokove.

Jedan od načina za čuvanje redova je da se za svaki red slike koristi po jedan BRAM blok kao što je prikazano na slici 4.5.



Slika 4.5. Realizacija linijskog bafera

Komponente linijskih bafera **RAM BUFFER** se mogu jednostavno opisati korišćenjem nizova u VHDL kodu, s tim što je potrebno voditi računa da se u svakom trenutku čita i upisuje najviše jedan element niza kako bi se niz sintetisao u okviru BRAM bloka. Širina jednog reda linijskog bafera odgovara maksimalno podržanoj širini slike i definiše se *generic* parametrom **C_MAX_IMG_WIDTH** koji se može zadati prilikom instanciranja komponente. Stvarna širina slike koja se obrađuje se definiše sadržajem registra **reg_img_w** i može se menjati i nakon instanciranja komponente (u *runtime*-u).

Jedan do načina realizacije linijskog bafera je da se koristi 2·FilterRadius RAM BUFFER-a, po jedan za svaki red slike koji je potrebno čuvati. U svakom taktu se iz svakog od blokova istovremeno čita po jedan podatak i upisuje podatak iz narednog reda. Na taj način se obezbeđuje da se na kraju procesiranja trenutnog reda slike vrši pomeraj prozora lokalnih susedstava na dole, tj u svaki bafer će biti iskopiran red koji se nalazi ispod reda koji je trenutno bio upisan, dok će se u prvom baferu nalaziti poslednji red slike koji je upisan u akcelerator.

Ako se pogleda situacija nakon pristizanja novog reda slike, sadržaj linijskog bafera je promenjen u samo jednom redu. Odnosno, umesto reda slike koji je najranije stigao upisan je najnoviji red slike. Dakle, umesto da se prilikom procesiranja upisuje u svaki red linijskog bafera, čime se obavlja kopiranje, dovoljno je vršiti upis u bafer koji sadrži najstariji red slike. U ovom slučaju je jedino potrebno voditi računa koji je to bafer reda u okviru linijskog bafera.

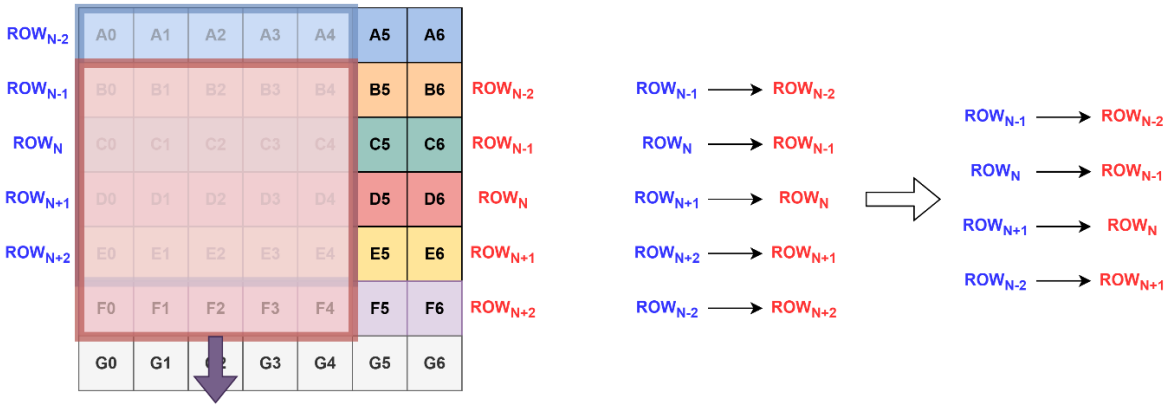
U slikama ispod je prikazan proces upisa novih podataka i čitanja iz linijskog bafera piksela potrebnih za procesiranje u okviru aritmetičke jedinice akceleratora na primeru FilterRadius=1.

Na slici 4.6. je prikazan proces čitanja prvih 2·FilterRadius redova slike. Na početku procesa filtriranja sve memorije linijskog bafera kao i registri aritmetičke jedinice su prazni, odnosno sadrže nevalidne podatke.

Slika 4.6. Čitanje/upisivanje podataka iz linijskog bafera i upis u aritmetičku jedinicu za prvih 2·FilterRadius redova slike

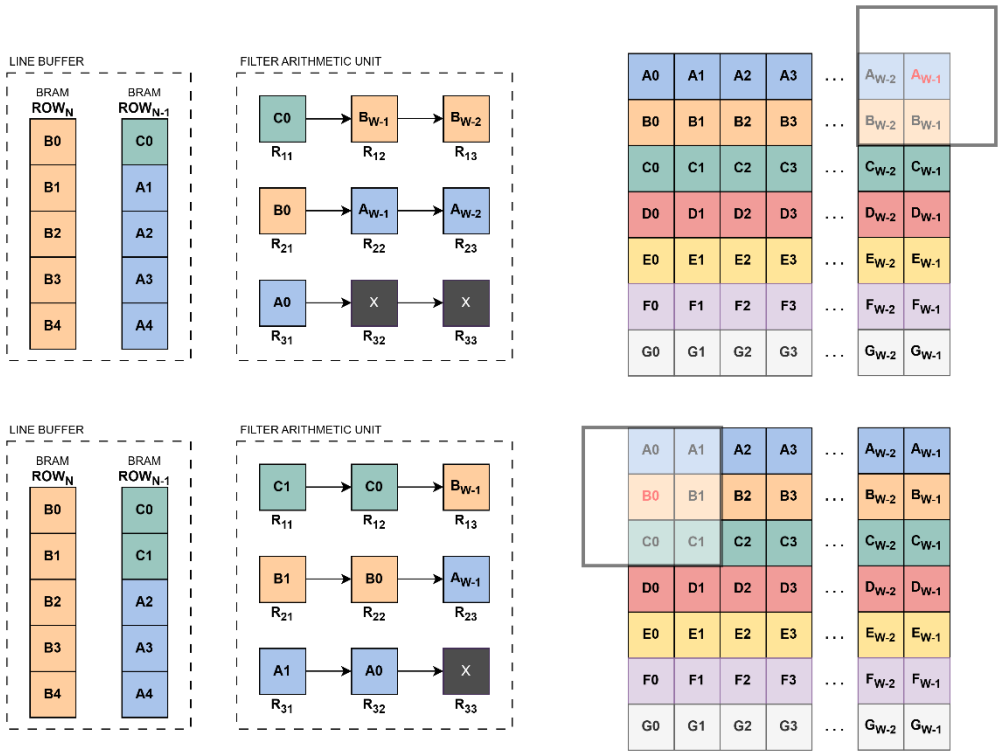
U akcelerator se najpre upisuju pikseli slike redom s leva na desno (pikseli od 0 do W-1 gde je W širina slike), odozgo na dole (redovi od A do kraja slike). Pikseli iz reda koji trenutno pristiže u akcelerator se upisuju u prvi red aritmetičke jedinice (registri $\mathbf{R_{11}}$, $\mathbf{R_{12}}$, ... , $\mathbf{R_{1K}}$). Pored upisa u registre aritmetičke jediniče, pristigli pikseli se upisuju i u linijski bafer kako bi bili dostupni za filtriranje narednih redova slike. Pristigli pikseli se upisuju u najstariji red linijskog bafera $\mathbf{ROW_{N\text{-}FilterRadius}}$ odnosno u primeru sa slike $\mathbf{ROW_{N\text{-}1}}$. Kada se upiše ceo red slike dolazi do

promene uloge bafera. Kako se sada menja vrednost trenutnog reda koji se procesira, odnosno N se inkrementira za 1, bafer **ROW_{N+C}** postaje **ROW_{N+C-1}** gde je C konstanta iz intervala [-2·FilterRadius, 2·FilterRadius-1] kao što je prikazano na slici 4.7. Ovde je potrebno obratiti pažnju da se prilikom procesiranja reda N u akcelerator upisuje red N+FilterRadius. Za ovaj red ne postoji poseban bafer već se pristigli pikseli direktno upisuju u aritmetičku jedinicu. Pošto red N-FilterRaidus neće biti potreban za procesiranje narednih redova slike novopristigli pikseli se upisuju u ovaj bafer na lokacije piskela koji su pročitani i upisanu u aritmetičku jedinicu. Dakle, na kraju procesiranja reda u baferu N-FilterRadius će se zapravi nalaziti elementi iz reda N+FilterRadius.



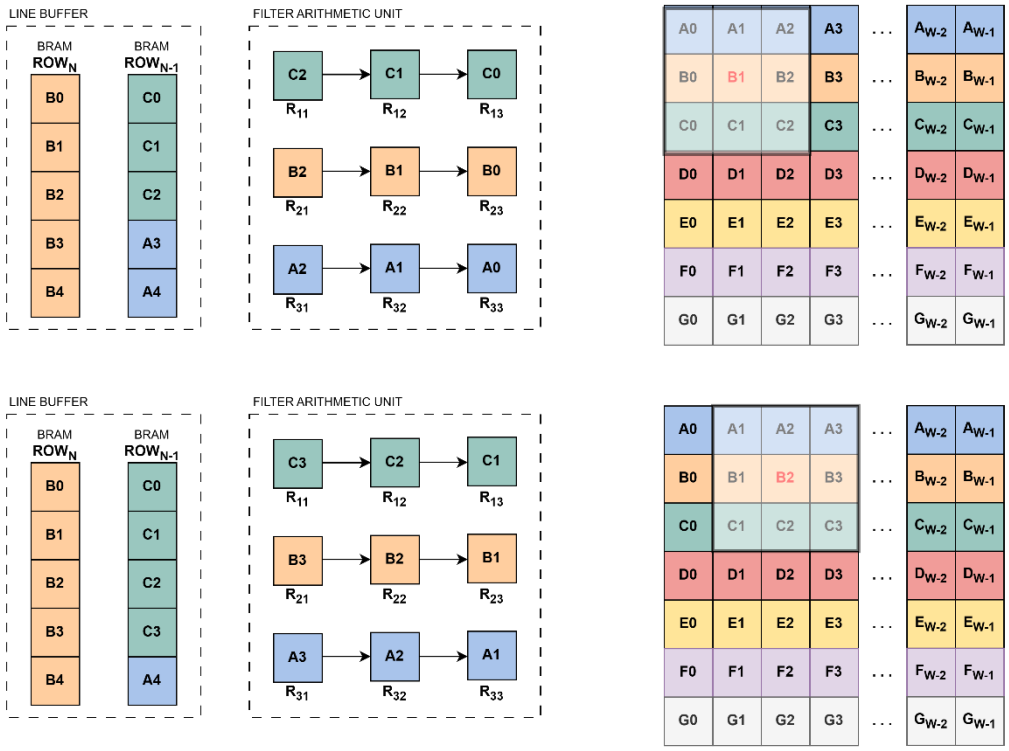
Slika 4.7. Pomeranje lokalnog susedstva na dole na kraju učitavanja celog reda na primeru filtra dimenzija 5x5, odnosno FilterRadius=2

Nakon što je učitan ceo red A obavlja se pomeranje redova odnsno promena pokazivača na linijske bafere, tako da red ROW_N postaje ROW_{N-1} a ROW_{N-1} preuzima ulogu bafera ROW_N. Sada u akcelerator pristižu pikseli iz reda B i oni se pored aritmetičke jedinice upisuju u bafer ROW_{N-1}. Istovremeno se u drugi red registara aritmetičke jedinice upisuju pikseli iz bafera ROW_N, odnosno pikseli reda A. Kao što se vidi na slici 4.6. dok god ne pristignu elementi reda C lokalno susedstvo za filtar radijusa 1 odnosno dimenzija 3x3 nije potpuno i rezultat još nije validan. Nakon upisa svih piksela iz reda B, baferi opet menjaju uloge i sada se elementi reda A nalaze u baferu ROW_{N-1} dok se elementi reda B nalaze u baferu ROW_N.



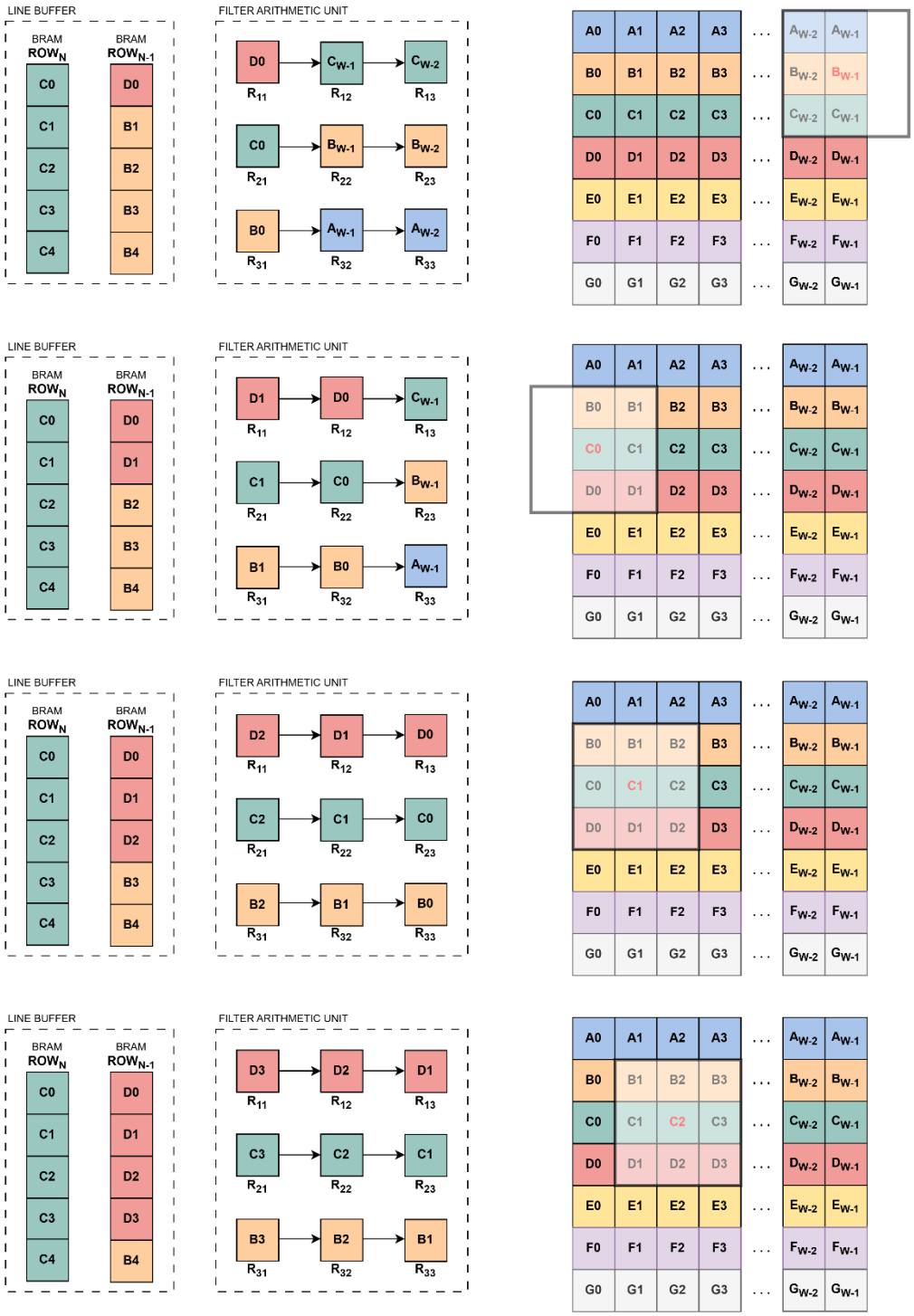
Slika 4.8. Nepotpuna lokalna susedstva na ivicama slike prilikom prelaska iz reda A u red B

Sada se u akceleratoru nalaze upisani redovi A i B. Ukoliko je radijus filtra 1 potrebni su još pikseli reda C za generisanje izlaza. Na slici 4.8. je prikazana situacija prilikom učitavanja prvih piksela reda C. Da bi se generisao prvi izlazni piksel potrebno je da stigne 2·FilterRadius+1 piksela reda C, odnosno 3 piksela u slučaju radijusa 1. Dakle, nakon upisa piksela C2 u akcelerator pristigli su pikseli iz lokalnog susedstva prvog piskela koji generiše izlaz. Na slici 4.9. je prikazana ova situacija i sadržaj registara aritmetičke jedinice u kojoj se nalaze pikseli iz lokalnog susedstva piksela koji se obrađuje. U svakom narednom taktu čita se po jedan piksel iz svakog reda linijskog bafera i upisuje u odgovarajuće registre artimetičke jedinice. Konkretno piksel pročitan iz bafera $\mathbf{ROW_{N+C}}$ se upisuje u registar $\mathbf{R_{FilterRadius-C+1\ 1}}$, odnosno za radijus fitra 1 ovo je prikazano na slici 4.5. tj pikseli iz bafera $\mathbf{ROW_N}$ se upisuju u registar $\mathbf{R_{21}}$, dok se pikseli iz bafera $\mathbf{ROW_{N-1}}$ upisuju u registar $\mathbf{R_{31}}$. Novopristigli pikseli u akcelerator uvek se upisuju u prvi red aritmetičke jedinice, odnosno registar $\mathbf{R_{11}}$, nevezano od radijusa filtra.



Slika 4.9. Popunjavanje lokalnog susedstva i kreiranje validnih izlaznih piksela za red B

Pikseli koji se nalaze na odstojanju manjem od radijusa filtra od ivica slike nemaju potpuno definisano lokalno susedstvo i, osim u slučaju kada je realizovano proširenje slike definisano bonusom, ne mogu generisati validne izlazne piksele. Ovakve situacije se dešavaju svaki put prilikom prelaska u naredni red pri čemu se validni izlazni pikseli generišu tek kada pristigne najmanje 2·FilterRadius + 1 piksela iz novog reda. Ova situacija je prikazana na slici 4.10. je za prelazak iz reda B u red C i radijus filtra koji iznosi 1.



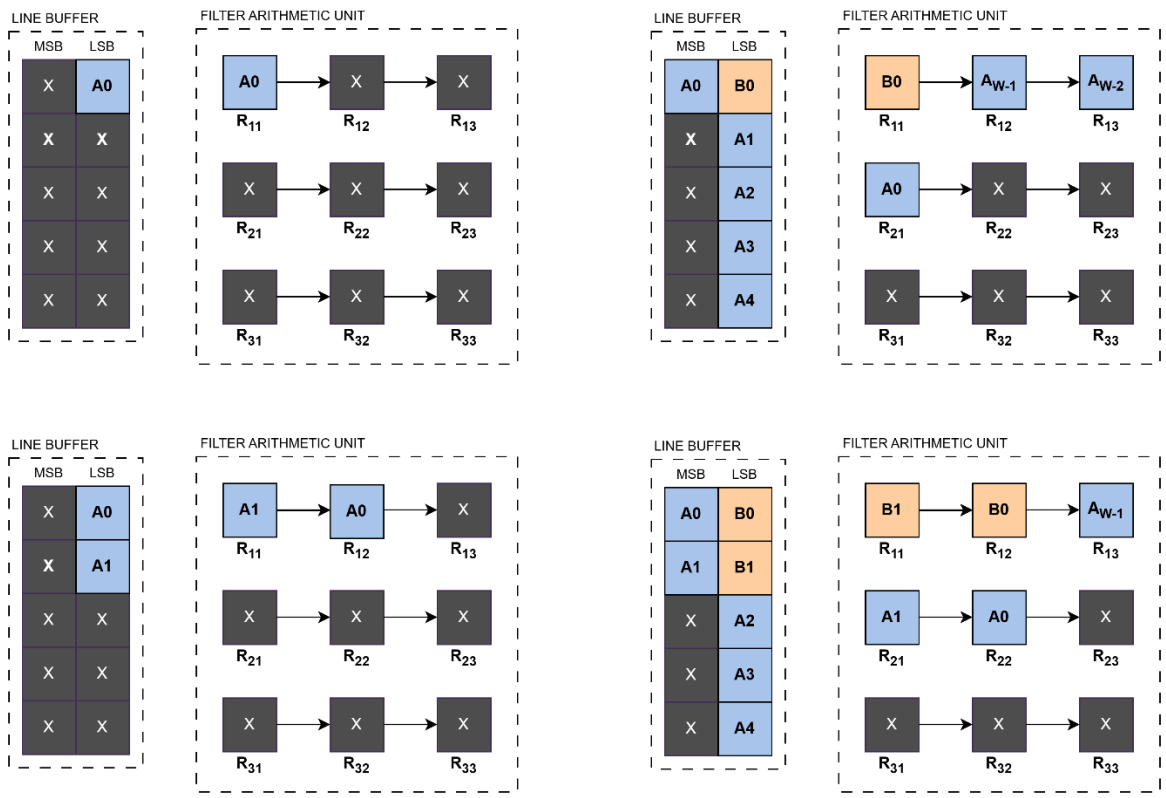
Slika 4.10. Formiranje validnog lokalnog susedstva prikom prelaska u naredni red, na primeru prelaska iz reda B u red C za radijus filtra 1

Kako se u svakom taktu u paraleli čitaju pikseli iz odgovarajućih redova potrebno je da svaki red bude sačuvan u posebnom bloku BRAM memorije. Broj ovih blokova je najmanje 2·MaxFilterRadius, širina memorijske reči je jednaka širini piksela, u ovom slučaju 8 bita, dok je dubina memorije jednaka maksimalno podržanoj širini slike, odnosno **C_MAX_IMG_WIDTH**. Memorijski blokovi koji se nalaze na Zynq 7000 čipu su kapaciteta 36Kb i mogu se konfigurisati sa različitim širinama memorijske reči. Opis blokova se nalazi u odgovarajućoj dokumentaciji. Svaki 36Kb BRAM blok se može podeliti na dva nezavisna 18Kb bloka. U slučaju širine memorijske reči 8 bita, koja odgovara jednom pikselu, u 18Kb bloku se

može čuvati 2048 piksela. Dakle, za implementaciju linijskog bafera za maksimalni podržan radijus filtra MaxFilterRadius potrebno je, u najgorem slučaju, sačuvati 2·MaxFilterRadius redova slike tj potrebno je 2·MaxFilterRadius memorijskih blokova. Kako se jedan 36Kb BRAM blok može podeliti na dva manja, ukoliko je širina slike manja od 2048 piksela, linijski bafer će zauzeti MaxFilterRadius 36Kb BRAM blokova. Ukoliko širina slike premaši 2048 broj BRAM blokova se duplira, ako premaši 4096 ponovo se duplira, itd.

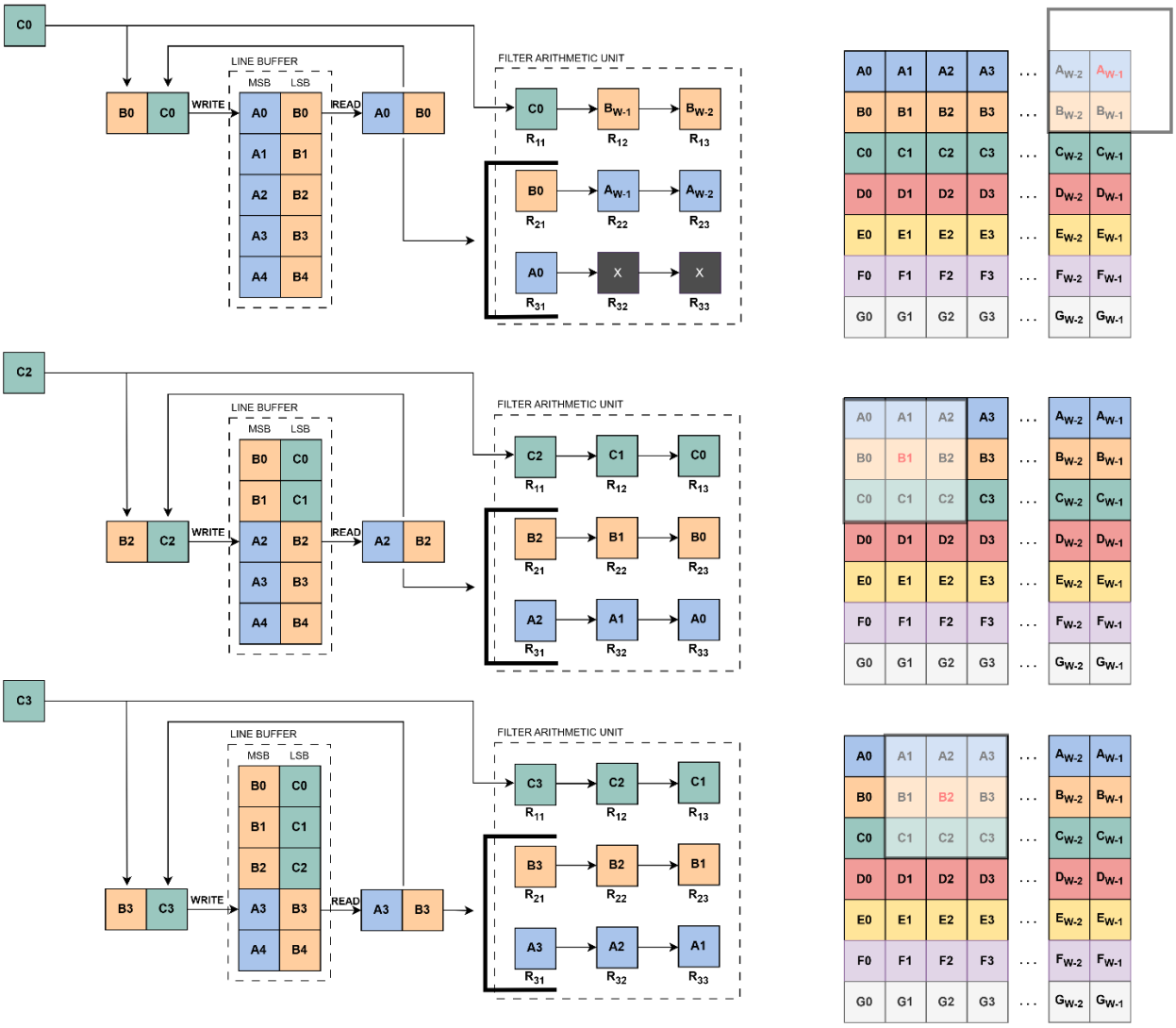
Problem sa ovakvim dizajnom je što se ne skalira dobro za manje dimenzije slike. Ukoliko na primer ograničimo maksimalnu širinu slike na 512 i maksimalni radijus filtra na 4 linijski bafer će i dalje zauzimati 4 36Kb BRAM bloka. Pri tome će svaki blok biti popunjen 25%, odnosno 512 od 2048 lokacija. Ostatak bloka se ne može koristiti za druge namene pošto je blok stalno zauzet odnosno u svakom taktu se čita odgovarajući podatak iz tog bloka. Na prvi pogled izgleda da je ovo optimalan dizajn, međutim ako se malo detaljnije analizira problem može se doći do boljeg rešenja.

Sa slike 4.5. se vidi da je prlikom pristizanja odgovarajućeg piksela u akcelerator potrebno pročitati sve piksele iz baferisanih redova koji se nalaze u istoj koloni kao piksel koji je upravo stigao. Kako RAM dozvoljava čitanje samo jednog podatka u jednom trenutku potrebno je svaki od ovih redova čuvati u posebnim blokovima kao što je i prethodno opisano. Ako se opet pogleda dokumentacija BRAM blokova, može se videti da širina memorijske reči nije ograničena na 8 bita već da se može fleksibilno definisati. Ukoliko bi memorijska reč bila dovoljno široka da u okviru nje stanu svi pikseli iz iste kolone lokalnog susedstva, onda bi svi mogli da se pročitaju u jednom taktu čak iako su smešteni u istom bloku BRAM memorije.



Slika 4.11. Korišćenje jedne memorijske reči za piksele iste kolone iz susednih redova. Najnoviji pikseli se uvek upisuju u najniži bajt memorijske reči.

Na slici 4.11. je prikazan postupak popunjavanja memorije prilikom učitavanja piksela slike iz prva dva reda za slučaj radijusa filtra 1. U tom slučaju je potrebno sačuvati piksele iz dva susedna reda slike pa je širina memorijske reči 16 bita, što je dovoljno za dva piksela. Najnoviji pikseli se upisuju u najniži bajt odgovarajuće memorijske reči čija adresa odgovara trenutnoj koloni, odnosno poziciji piksela u redu. Po učitavanju novog reda (druga kolona na slici 4.11.) najpre se čita reč sa odgovarajuće adrese u memoriji, koja odgovara trenutnoj koloni. Ova reč se pomera za 8 bita u levo, što znači da pikseli koji odgovaraju najstarijem redu ispadaju iz memorije, dok se pikseli najnovijeg reda upisuju na poziciju najnižeg bajta memorijske reči.

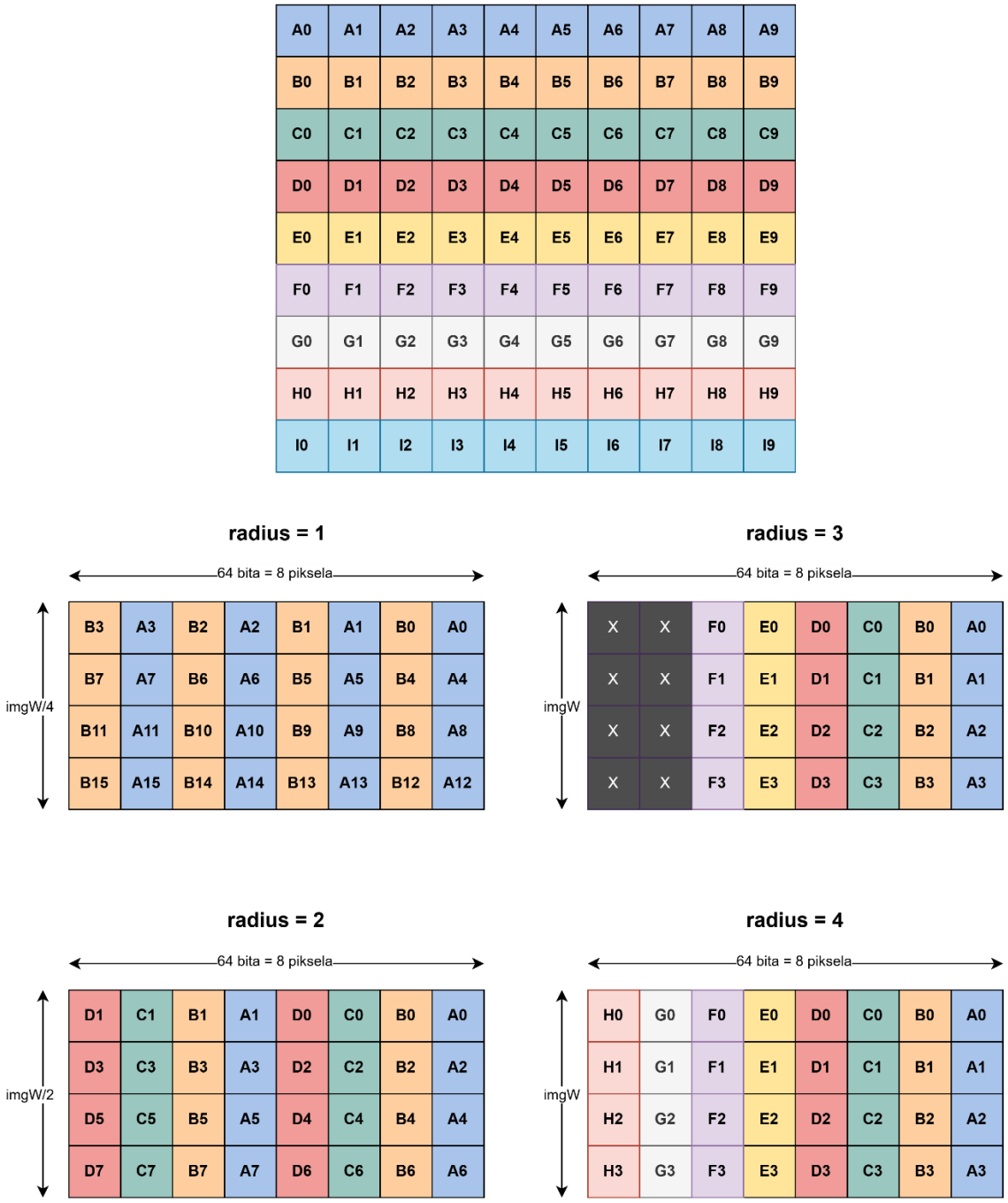


Slika 4.12. Formiranje validnog lokalnog susedstva prikom prelaska u naredni red, na primeru prelaska iz reda B u red C za radijus filtra 1, za slučaj kada se cela kolona lokalnog susedstva čuva u jednoj memorijskoj reči.

Na slici 4.12. je prikazano formiranje validnog lokalnog susedstva potrebnog za generisanje izlaznih rezultata. Vidi se da se u registar $\mathbf{R_{11}}$ aritmetičke jedinice upisuje podatak koji je pristigao sa AXI Stream ulaza dok se ostali registri $\mathbf{R_{21}}$, $\mathbf{R_{31}}$,... popunjavaju podacima iz jedne memorijske reči linijskog bafera. Na ovaj način se obezbeđuje bolje iskorišćenje BRAM modula, pogotovo za slučaj manjih širina slike. Na primer, u slučaju filtra radijusa 4, potrebno je sačuvati piksele iz 8 redova, tj širina memorijske reči će biti 64 bita. Ukoliko je širina slike manja od 512 biće iskorišćen svega jedan 36kb BRAM blok, dok se u slučaju slike manje od

256 piksela linijski bafer može implementirati pomoću jednog 18kb odnosno pola 36kb BRAM bloka. Demonstrirati uspešnu realizaciju ovog dela zadatka prikazom zauzeća BRAM blokova.

[BONUS 2 → +5 poena] Ukoliko je širina filtra uvek ista onda je organizacija memorije jasna, odnosno širina memorijske reči je jednaka dvostrukom radijusu filtra. U slučaju filtra radijusa 4 koji je zahtevan u ovom projektu, širina memorijske reči je 64-bita i u njoj se mogu čuvati pikseli iz 8 susednih redova. U tom slučaju jedan 36kb BRAM blok može podržati slike maksimalne širine 512 piksela. Međutim ukoliko se u toku rada odabere filtar manjih dimenzija, recimo radijusa 1, memorijska reč neće biti opitmalno korišćena.

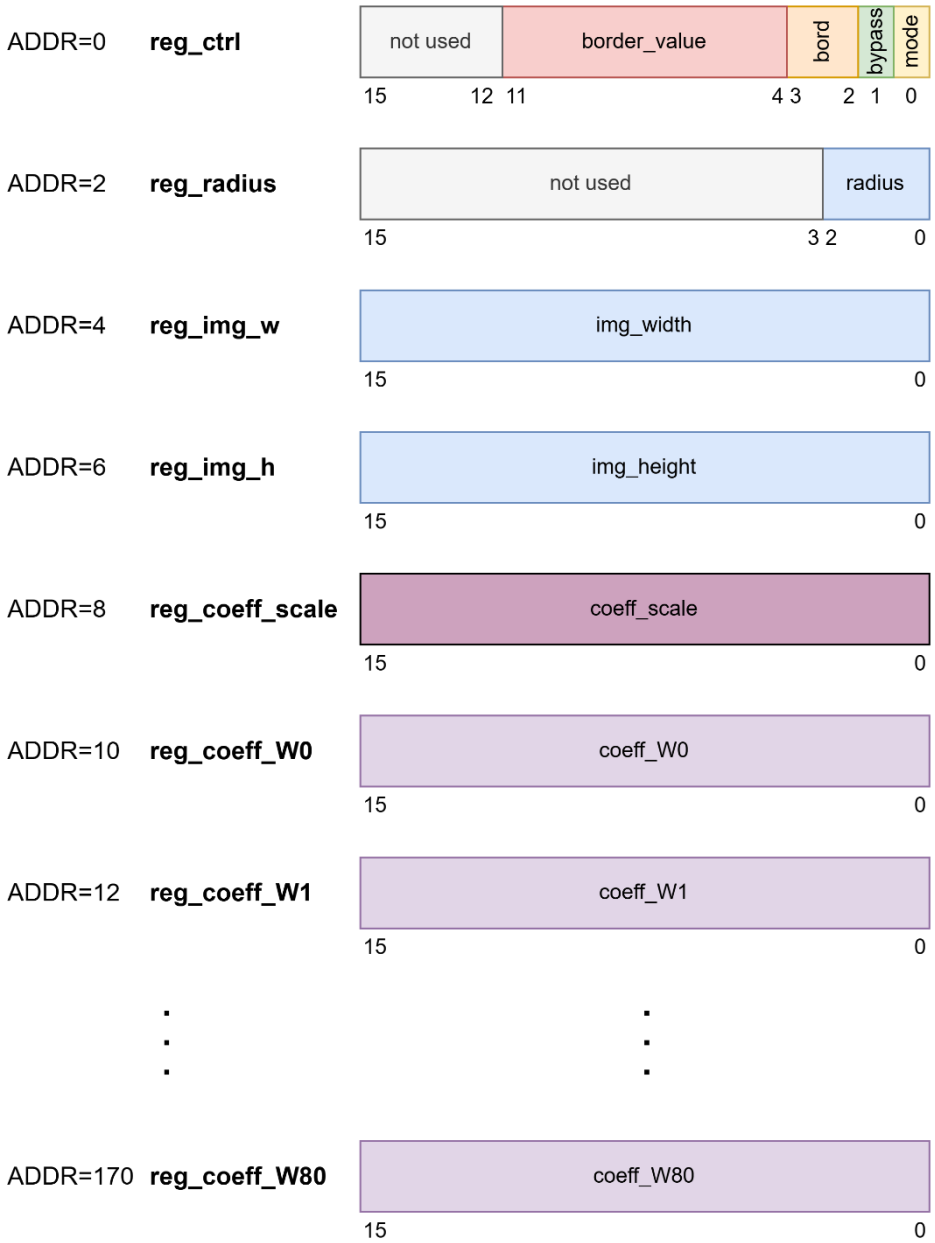


Slika 4.13. Memorijska organizacija linijskog bafera za različite širine filtra u slučaju da je maksimalno podržan filtar radijusa 4.

Na slici 4.13. je prikazana memorijska organizacija linijskog bafera kojom se omogućava proširenje maksimalne širine filtrirane slike, bez povećanja broja memorijskih blokova, ukoliko se koristi filtar manjeg radijusa. Na ovaj način se dodatno smajuje i potrošnja pošto se smanjuje učestanost čitanja iz linijskog bafera. Demonstrirati uspešnu realizaciju ovog dela zadatka prikazom zauzeća BRAM blokova, i mogućnost obrade većih širina slike za manje radijusa filtra. **[BONUS 2]**

5. Registri modula akceleratora

Potrebno je obezbediti da projektovani modul akceleratora ima 5 registara za parametre slike i filtra, plus 81 registar za koeficijente filtra. Svi registri su širine 16 bita. Adresna mapa i opis registara su prikazani na slici 5.1. i u Tabeli 5.1.



Slika 5.1. Prikaz registara modula akceleratora

Tabela 5.1. Adresna mapa i opis registara akceleratorskog modula

Adresa	Naziv registra	Opis
0x0	**reg_ctrl**	Kontrolni registar akceleratora. Sadržaj definisan u Tabeli 5.2.
0x2	**reg_radius**	Vrednost radijusa filtra. Filtar ima jednaku visinu i širinu koja iznosi 2·radius+1. Najniža 3-bita registra definišu vrednost radijusa iz opsega [0,4] ostali biti registra se ne koriste.
0x4	**reg_img_w**	Širina ulazne slike. Zadaje se kao 16 bitni neoznačeni ceo broj.
0x6	**reg_img_h**	Visina ulazne slike. Zadaje se kao 16 bitni neoznačeni ceo broj.
0x8	**reg_coeff_scale**	Faktor skaliranja koeficijenata. Ovim faktorom je potrebno skalirati izlazni rezultat filtriranja pre slanja na izlaz. Predstavlja neoznačeni 16-bitni podatak gde viša 4 bita predstavljaju ceo deo dok nižih 12 bita predstavlja razlomljeni deo faktora skaliranja.
0x10	**reg_coeff_W0**	Koeficijent W0 filtra zadat kao 16 bitni označeni broj sa 1 bitom za ceo deo i 15 bita za razlomljeni deo.
0x12	**reg_coeff_W1**	Koeficijent W1 filtra zadat kao 16 bitni označeni broj sa 1 bitom za ceo deo i 15 bita za razlomljeni deo.
. . .	. . .	. . .
0x170	**reg_coeff_W80**	Koeficijent W80 filtra zadat kao 16 bitni označeni broj sa 1 bitom za ceo deo i 15 bita za razlomljeni deo.

Tabela 5.2. Sadržaj bita kontrolnog registra (**reg_ctrl**)

Bit	Naziv	Opis
15..12	N/A	Ne koriste se.
0	**MODE**	Definiše koja vrednost se šalje na izlaz modula: **0** – Izlazna vrednost je neoznačeni 8-bitni podatak **1** – Izlazna vrednost je označeni 16-bitni podatak pri čemu se 9 bita koristi za ceo deo a 7 bita za razlomljeni deo.
1	**BYPASS**	Definiše da li se na izlaz samo prosleđuju elementi ulazne slike. Na taj način se preskače celokupno procesiranje akceleratora. **0** – Na izlaz se prosleđuje vrednost definisana MODE bitima **1** – Na izlaz se prosleđuju vrednosti ulaznih piksela
3..2	**BORD**	Definiše način na koji način se tretiraju elementi koji figurišu prilikom izračunavanja gradijenta ali izlaze van okvira slike. **0x** – Ne obrađuju se elementi koji nemaju potpuno lokalno susedstvo. **10** – Elementi koji izlaze van okvira slike imaju vrednost najbližeg elementa koji se nalazi u slici. **11** – Elementi koji izlaze van okvira slike imaju vrednost definisanu BORDER_VALUE vrednošću.
11..4	**BORDER_VALUE**	Definiše 8-bitnu vrednost koja se koristi za elemente lokalnog susedstva koji izlaze van okvira slike u slučaju kada opcija proširenja BORD ima vrednost 11.
15..12	N/A	Ne koriste se.

Obezbediti da se svakom od registara može pristupiti sa procesorskog dela sistema korišćenjem AXI Lite memorijski mapirane magistrale **s_axi_ctrl**. **Obezbediti da se vrednosti**

parametara procesiranja mogu promeniti jedino u slučaju da trenutno nije u toku proces obrade, pri čemu se u odgovarajuće registre može upisati u bilo kom trenutku.

Slika 5.2. Mapa koeficijenta filtra koji se koriste za različite vrednosti radijusa filtra.

6. Kreiranje koeficijenata filtra i konfiguracija

Kreirani akcelerator može da podrži proizvoljne linearne, prostorno invarijantane filtre neparnih dimenzija do maksimalnih dimenzija 9x9, odnosno radijusa 4. Filtri se zadaju parametrom radijusa tako da su svi neparnih dimenzija, odnosno moguće dimenzije filtra su 1x1, 3x3, 5x5, 7x7 i 9x9. Koeficijenti filtra su predstavljeni kao 16 bitni označeni brojevi sa fiksnom tačkom kod kojih 1 bit predstavlja ceo deo i znak, dok 15 bita predstavlja razlomljeni deo. Dakle opseg vrednosti koje koeficijenti filtra mogu uzimati je [-1, 0.99997]. Opseg koeficijenata se može proširiti korišćenjem faktora skaliranja (**coeff_scale**) koji predstavlja 16 bitni neoznačeni broj sa fiksnom tačkom kod koga je 4 bita odvojeno za ceo deo dok se 12 bita koristi za razlomljeni deo.

Koeficijenti filtra koji se ubrzava pomoću akceleratora mogu ispadati izvan opsega [-1, 0.99997] ili mogu biti skoncentrisani u malom delu opsega čime se povećava greška kvantizacije. Faktor skaliranja služi da opseg koeficijenata filtra prilagodi što je bolje moguće opsegu koji koristi akcelerator kako bi se u procesu kvantizacije napravila što manja greška. Postupak podrazumeva da se koeficijenti originalnog filtra pre kvantizacije pomnože inverznom vrednošću faktora skaliranja. Na kraju procesa filtriranja se rezultat množi faktorom skaliranja čime se poništava prethodno množenje inverznom vrednošću i generiše ispravan rezultat. Voditi računa da, prilikom množenja koeficijenata originalnog filtra inverznim faktorom skaliranja, nove vrednosti ne izađu izvan opsega kojim su predstvljeni koeficijenti filtra u akceleratoru.

U nastavku će biti prikazano par primera za generisanje koeficijenata filtra za konfiguraciju akceleratora.

Prvi primer predstavlja idealni filtar (box filtar) za usrednjavanje dimenzija 9x9. Svi koeficijenti ovog filtra imaju jednake vrednosti koje iznose 1/81. Predstavljanje ovog podatka sa 15 bita u razlomljenom delu podrazumeva množenje podatka sa 2^{15} i zaokruživanje na najbliži ceo broj. U ovom slučaju $(1/81)\cdot 2^{15} = 404.54$ odnosno 405 nakon zaokruživanja. Ovo je vrednost koju je potrebno upisati u sve registre koeficijenata akceleratora. To znači da svaki koeficijent umesto prave vrednosti 1/81 predstavlja vrednost $1/(405/2^{15})$ odnosno 1/80.9. Greška koja se pravi na svakom koeficijentu iznosi $1.39\cdot 10^{-5}$. Faktor skaliranja u ovom slučaju je 1, međutim pošto ovaj podatak ima 12 razlomljenih bita, u odgovarajući registar akceleratora je potrebno upisati vrednost $1\cdot 2^{12} = 4096$.

405	405	405	405	405	405	405	405	405
405	405	405	405	405	405	405	405	405
405	405	405	405	405	405	405	405	405
405	405	405	405	405	405	405	405	405
405	405	405	405	405	405	405	405	405
405	405	405	405	405	405	405	405	405
405	405	405	405	405	405	405	405	405
405	405	405	405	405	405	405	405	405
405	405	405	405	405	405	405	405	405

Slika 6.1. Koeficijenti idealnog filtra za usrednjavanje (box filtra) za radijus R=4 i faktor skaliranja 1.

Ova greška se može dodatno smanjiti ukoliko se iskoristi faktor skaliranja. Ako se za vrednost inverznog faktora skaliranja (vrednost kojom se množe koeficijenti filtra) postavi vrednost 3, koeficijenti filtra postaju 1/27 odnosno kada se predstave na 15 razlomljenih bita $(1/27)\cdot 2^{15} = 1213.63$ odnosno 1214 nakon zaokruživanja. Ova vrednost se upisuje u registre koeficijenata akceleratora. Vrednost koeficijenata filtra, nakon skaliranja i zaokruživanja je sada $1/(1214/2^{15})$ odnosno 1/26.992. Da bi se dobio ispravan rezultat, finalnu vrednost filtriranja je potrebno pomnožiti faktorom skaliranja koji iznosi 1/3 predstavljenim 12 razlomljenih bita odnosno $(1/3)\cdot 2^{12} = 1365.33$ tj 1365 nakon zaokruživanja odnosno 1/3.0007. U registar faktora skaliranja akceleratora je dakle potrebno upisati vrednost 1365. Greška koja se pravi na svakom koeficijentu, u ovom slučaju, je značajno manja i iznosi $7.53\cdot 10^{-7}$.

1214	1214	1214	1214	1214	1214	1214	1214	1214
1214	1214	1214	1214	1214	1214	1214	1214	1214
1214	1214	1214	1214	1214	1214	1214	1214	1214
1214	1214	1214	1214	1214	1214	1214	1214	1214
1214	1214	1214	1214	1214	1214	1214	1214	1214
1214	1214	1214	1214	1214	1214	1214	1214	1214
1214	1214	1214	1214	1214	1214	1214	1214	1214
1214	1214	1214	1214	1214	1214	1214	1214	1214
1214	1214	1214	1214	1214	1214	1214	1214	1214

Slika 6.2. Koeficijenti idealnog filtra za usrednjavanje (box filtra) za radijus R=4 i faktor skaliranja 1/3.

Drugi primer filtra je Gausov filtar za usrednjavanje. Koeficijenti filtra su zadati sledećim izrazom:

$$W_{Gauss}(x,y) = \begin{cases} \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}, |x| \leq R, |y| \leq R \\ 0, u\ ostalim\ slučajevima \end{cases}$$

Gde su *x* i *y* odstojanja koeficijenta filtra prostorne maske od centralnog elementa maske, σ standardna devijacija Gausove funkcije a *R* radijus filtra. Nakon zadavanja vrednosti standardne devijacije i radijusa filtra određuju se vrednosti koeficijenata na osnovu prethodnog izraza. Finalne vrednosti koeficijenata se dobijaju deljenjem svakog koeficjenta sumom svih koefcijenata filtra. Na taj način se postiže da je suma svih koeficijenata filtra jednaka 1 kako bi filtar zaista predstavljao prostorno usrednjavanje. Na primer, ukoliko se odabere filtar standardne devijacije σ=1, radijusa R=4 i faktor skaliranja 1 dobijaju se koeficijenti filtra (nakon množenja sa 2^{15}) koje je potrebno upisati u registre akceleratora:

0	0	0	1	2	1	0	0	0
0	1	8	35	58	35	8	1	0
0	8	96	428	706	428	96	8	0
1	35	428	1919	3163	1919	428	35	1
2	58	706	3163	5215	3163	706	58	2
1	35	428	1919	3163	1919	428	35	1
0	8	96	428	706	428	96	8	0
0	1	8	35	58	35	8	1	0
0	0	0	1	2	1	0	0	0

Slika 6.3. Koeficijenti Gausovog filtra za usrednjavanje za radijus R=4, standardnu devijaciju σ=1 i faktor skaliranja 1.

Maksimalna greška koja se pravi kvantizacijom koeficijenata na ovaj način iznosi $1.46 \cdot 10^{-5}$. Greška se može dodatno smanjiti korišćenjem faktora skaliranja. Ako se vrednost inverznog faktora skaliranja postavi na 4 dobijaju se koeficijenti filtra prikazani na slici 6.4. Da bi se dobio ispravan rezultat finalnu vrednost filtriranja je potrebno pomnožiti faktorom skaliranja koji iznosi 1/4 predstavljenim 12 razlomljenih bita odnosno $(1/4) \cdot 2^{12} = 512$. Greška koja se pravi na svakom koeficijentu, u ovom slučaju, je nešto manja i iznosi $3.36 \cdot 10^{-6}$.

0	0	1	4	7	4	1	0	0
0	3	31	141	232	141	31	3	0
1	31	382	1712	2823	1712	382	31	1
4	141	1712	7674	12653	7674	1712	141	4
7	232	2823	12653	20861	12653	2823	232	7
4	141	1712	7674	12653	7674	1712	141	4
1	31	382	1712	2823	1712	382	31	1
0	3	31	141	232	141	31	3	0
0	0	1	4	7	4	1	0	0

Slika 6.4. Koeficijenti Gausovog filtra za usrednjavanje za radijus R=4, standardnu devijaciju σ=1 i faktor skaliranja 1/4.

Dalje povećanje inverznog faktora skaliranja bi omogućilo preciznije predstavljanje manjih koeficijenata na obodima maske ali može dovesti do prekoračenje koeficijenata koji imaju veće vrednosti (na primer centrslni koeficijent).

Treći primer filtra je LoG (*Laplacian of Gaussian*) koji se koristi za detekciju diskontinuiteta. Koeficijenti filtra su zadati sledećim izrazom:

$$W_{LoG}(x,y) = \begin{cases} -\frac{1}{\pi\sigma^4}\left[1-\frac{x^2+y^2}{2\sigma^2}\right]e^{-\frac{x^2+y^2}{2\sigma^2}}, |x| \le R, |y| \le R \\ 0, u\ ostalim\ slučajevima \end{cases}$$

Na primer, ukoliko se odabere filtar standardne devijacije σ=1, radijusa R=4 i faktor skaliranja 1 dobijaju se koeficijenti filtra (nakon množenja sa 2^{15}) koje je potrebno upisati u registre akceleratora:

0	0	4	16	24	16	4	0	0
0	10	86	281	406	281	86	10	0
4	86	573	1284	1412	1284	573	86	4
16	281	1284	0	-3163	0	1284	281	16
24	406	1412	-3163	-10430	-3163	1412	406	24
16	281	1284	0	-3163	0	1284	281	16
4	86	573	1284	1412	1284	573	86	4
0	10	86	281	406	281	86	10	0
0	0	4	16	24	16	4	0	0

Slika 6.5. Koeficijenti LoG filtra za radijus R=4, standardnu devijaciju σ=1 i faktor skaliranja 1.

Maksimalna greška koja se pravi kvantizacijom koeficijenata na ovaj način iznosi $1.5\cdot10^{-5}$. Greška se može dodatno smanjiti korišćenjem faktora skaliranja. Ako se vrednost inverznog faktora skaliranja postavi na 2 maksimalna greška kvantizacije spada na $7.6\cdot10^{-6}$.

Četvrti primer predstavlja filtar za izoštravanje slike. Ovi filtri se dobijaju kada se od originalne slike oduzme rezultat filtriranja filtrom za usrednjavanje. Rezultat razlike predstavlja detalje sa slike. Ovi detalji se onda množe faktorom izoštravanja **k** i sabiraju sa ulaznom slikom, čime se dobija izoštrena slika. Opšti izraz za koeficijent filtra za izoštravanje je:

$$W_{Sharpen}(x,y) = (1+k)\cdot\delta(x,y) - k\cdot W_{LowPass}(x,y)$$

Pri čemu Dirakova funkcija δ(x,y) predstavlja matricu dimenzija (2R+1)×(2R+1) gde je R radijus filtra pri čemu je δ(0,0) jednaka 1 dok su sve ostali elementi ove matrice jednaki 0. Filtar za usrednjavanje $W_{LowPass}$ može biti različitog tipa, na primer idealni (box) filtar ili Gausov filtar za usrednjavanje.

-202	-202	-202	-202	-202	-202	-202	-202	-202
-202	-202	-202	-202	-202	-202	-202	-202	-202
-202	-202	-202	-202	-202	-202	-202	-202	-202
-202	-202	-202	-202	-202	-202	-202	-202	-202
-202	-202	-202	-202	32566	-202	-202	-202	-202
-202	-202	-202	-202	-202	-202	-202	-202	-202
-202	-202	-202	-202	-202	-202	-202	-202	-202
-202	-202	-202	-202	-202	-202	-202	-202	-202
-202	-202	-202	-202	-202	-202	-202	-202	-202

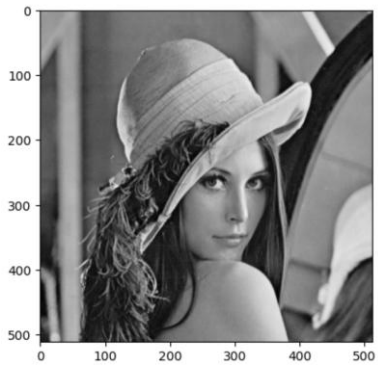
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	-1820	-1820	-1820	0	0	0
0	0	0	-1820	30948	-1820	0	0	0
0	0	0	-1820	-1820	-1820	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Slika 6.6. Koeficijenti filtra za izoštravanje baziranog na box filtru za radijus R=4 (gore) i radijus R=1 (dole), faktor skaliranja 2 i intenzitet izoštravanja k=1.

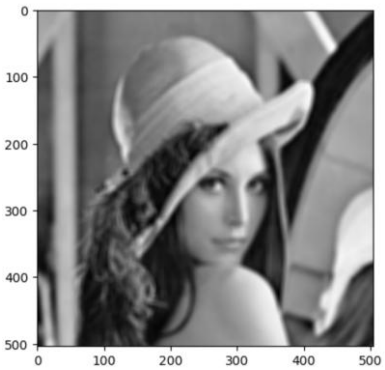
0	0	0	-1	-1	-1	0	0	0
0	0	-4	-18	-29	-18	-4	0	0
0	-4	-48	-214	-353	-214	-48	-4	0
-1	-18	-214	-959	-1582	-959	-214	-18	-1
-1	-29	-353	-1582	30160	-1582	-353	-29	-1
-1	-18	-214	-959	-1582	-959	-214	-18	-1
0	-4	-48	-214	-353	-214	-48	-4	0
0	0	-4	-18	-29	-18	-4	0	0
0	0	0	-1	-1	-1	0	0	0

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	-49	-218	-359	-218	-49	0	0
0	0	-218	-977	-1611	-977	-218	0	0
0	0	-359	-1611	30112	-1611	-359	0	0
0	0	-218	-977	-1611	-977	-218	0	0
0	0	-49	-218	-359	-218	-49	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

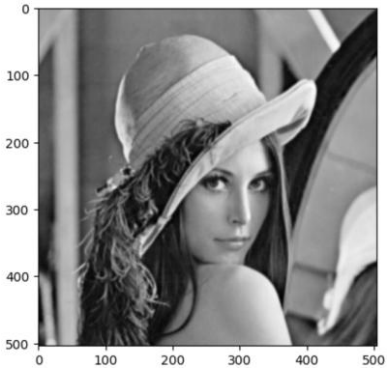
Slika 6.7. Koeficijenti filtra za izoštravanje baziranog na Gausovom filtru za radijus R=4 (gore) i radijus R=2 (dole), standardnu devijaciju Gausovog filtra σ=1, faktor skaliranja 2 i intenzitet izoštravanja k=1.



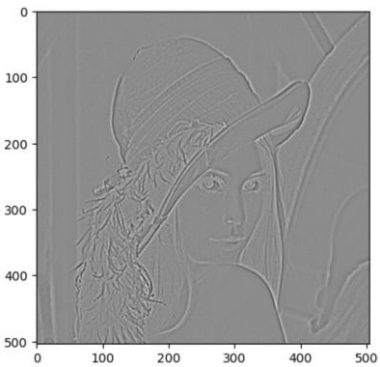
a) Ulazna slika lena_512.bin



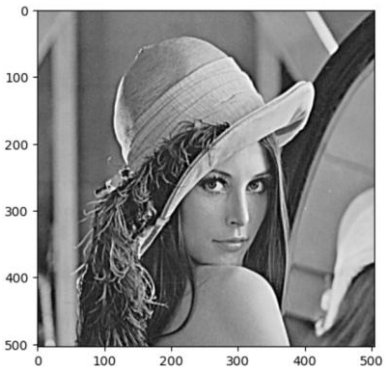
b) Idealni (box) filtar R=4



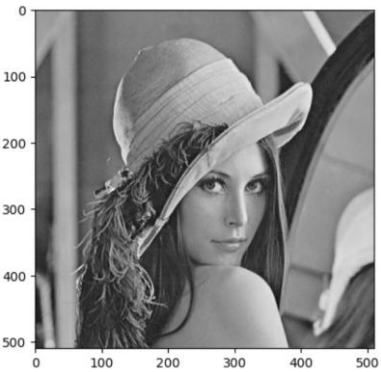
c) Gausov filtar R=4, σ=1



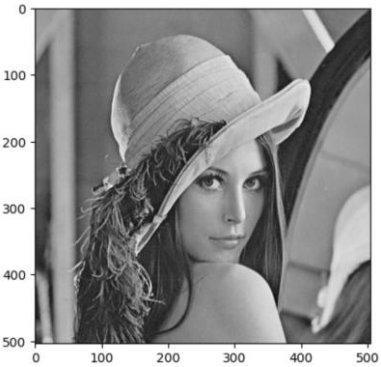
d) LoG filtar R=4, σ=1



e) Filtar za izoštravanje na bazi box filtra R=4, k=1



f) Filtar za izoštravanje na bazi box filtra R=1, k=1



d) Filtar za izoštravanje na bazi Gausovog filtra R=4, σ=1, k=1

Slika 6.8. Rezultat primene prikazanih filtara na slci lena_512.bin dimenzija 512x512

7. Izveštaj

Na kraju projekta je potrebno da kreirati izveštaj koji predstavlja dokumentaciju projekta i opisuje arhitekturu sistema, realizaciju hardvera i softvera za svaki od delova projekta. Izveštaj predstavlja dokumentaciju projektovanog sistema i potrebno je da bude dovoljno detaljan kako bi neko uz izveštaj i kod mogao potpuno da reprodukuje rezultate i iskoristi delove projekta u većem sistemu. U izveštaju je potrebno opisati testiranje sistema, prikazati dobijene rezultate filtriranja, zauzeće resursa kao i izmerena vremena izvršavanja i postignuta ubrzanja.

Za pun broj poena potrebno je da pored potpune funkcionalnosti sistem bude smisleno particionisan i kod uredno napisan sa dovoljnim brojem komentara.

Preporuka je da na početku rada na projektu kreirate svoj blog gde ćete opisivati delove sistema koje ste implementirali, rezultate koje ste postigli, probleme koje ste rešili i uvide koje ste stekli. Blog nazovite **DVS25_G[broj grupe]_[slobodan naziv]** na primer **DVS25_G0_Akcelerator_filtriranja_slike**. Prednost bloga je što vam može poslužiti kasnije kao referenca, lako ga možete podeliti, pokazujete kontinuitet u radu a i omogućava da na kraju relativno lako sastavite izveštaj pošto će sve relevantne informacije biti na jednom mestu. Portali na kojima se može besplatno pokrenuti blog su https://www.blogger.com/ i https://wordpress.org/ ili bilo koja druga platforma koja vam najviše odgovara. Po otvaranju bloga bilo bi lepo da pošaljete link i predmetnom nastavniku/asistentu kako bi mogli da prate progres projekta. **Važno je da redovno osvežavate vaš blog!**

8. Slanje rešenja

Prilikom slanja rešenja potrebno je kompletan projekat zapakovati u arhivu korišćenjem opcije *File->Project->Archive*. Zapakovati u arhivu i kompletan workspace softverskog dela sistema.

Rešenje projekta se sastoji iz: (1) **arhive hardverskog dela**, (2) **arhive softverskog dela**, (3) **izveštaja**, (4) **dodatnih korišćenih slika** i (5) **python skripte**. Navedene fajlove zapakovati u novu arhivu **dvs25_projekat_g[X].zip** i postaviti ovu arhivu na fakultetski OneDrive.

Rešenja projekta slati sa naslovom **[DVS] Projekat - GX** . U telu poruke navesti imena i brojeve indeksa članova grupe kao i link ka rešenju projekta zapakovanog na prethodno opisani način.

Projekat se može braniti u svakom ispitnom roku.

Da bi ocena bila upisana u odgovarajućem ispitnom roku potrebno je projekat predati najkasnije do termina ispita u tom ispitnom roku.